Inventory boundary review — 24 September 2026

Selected skill: `010-aqstoqflow-inventory-boundary-gate`. Scope: the original 20 stock mutation findings, two previously marker-exempt fixture scripts, the scanner, and shared disposable fixture guards. No database connection, write, migration, sale, charge, refund, or browser fixture execution was performed.

Final result: inventory boundary `fail` mode passes, exit 0, with 0 active findings at 09:00:33 UTC. The 55 detected calls comprise 19 kernel calls, 8 pre-existing Prisma seed calls, 4 test calls (including three new mocked guard invocations), and 24 individually reviewed disposable fixture calls. This clears this engineering boundary check only; it does not approve G6 inventory business policy or a POS release.

Evidence baseline

- The original pre-edit repository scan could not save its output because the filesystem denied the evidence write (`EPERM`). It is not reported as a successful fresh baseline.
- The historical 23 September report records 20 active findings and 72 line-pattern matches. Those remain historical figures.
- The first successfully saved current scan (`evidence/inventory-before.md/json`, generated 08:51:41 UTC) uses the hardened AST scanner before fixture classifications: **24 active, 28 allowed, 52 actual calls**. Four additional active writes were previously waived by demo comments. The AST scan omits comments and string examples, so its total is intentionally not comparable to the old text-match total.
- This first saved scan already includes the guard edits. It is a pre-classification result, not an untouched-source baseline. Mode is `fail`, exit 1.

Findings and fixes

1. **A demo comment could waive an arbitrary operational script.** Removed the `@inventory-boundary-demo-script` exemption. A captured pre-edit scanner from HEAD reproduces the bypass; the actual gate diff confirms this was the pre-edit implementation. The regression expected failure but received success.
2. **Line-based matching missed multiline, literal bracket, and optional-chain writes and counted inert text.** The scanner now parses real call expressions with the existing TypeScript dependency. A pre-edit regression detects only two inert text examples while missing three actual writes; the hardened scanner detects the three writes and ignores inert examples.
3. **The stock booster used the application's default Prisma URL.** It now requires `POS_DEMO_FIXTURE_DATABASE_URL`, exact local database `stoquify_pos_demo_disposable`, non-production environment, reviewed connection parameters, and actual connected database identity before work. It does not provision this database.
4. **The supplier PO fixture accepted any local application database and an optional nonlocal override.** It now requires `SUPPLIER_PO_ACK_FIXTURE_DATABASE_URL` for exact local `stoquify_supplier_po_ack_disposable`; the shared guard rejects production and redirect parameters, and connected identity is checked before seeding or assurance work. This intentionally requires updating isolated test setup before rerunning this fixture. It does not create or migrate a database.
5. **The historical fixtures were synthetic but exported fixture functions could be called without the caller's ownership preflight.** The shared Prisma client now verifies connected identity and existing

synthetic ownership before its first model operation. The environment is validated before it is set to `test`. Each of the eleven entrypoints was executed in an isolated VM with an application target and mocked dependencies: every entrypoint rejected before constructing Prisma. Separate tests invoke the shared model operation directly and prove connected identity/ownership failures prevent the write callback.

6. **The modern module fixture already had strong transaction identity and organization provenance checks.** Added protocol/connection-parameter validation before its existing exact-name guard. Its imported function was exercised with a mismatched connected transaction identity; organization and stock mutation callbacks were never reached.

Individual callsite review

All 24 current callsites, including the original 20, are recorded with file, line, mutation, business purpose, exact source hash, guard dependencies, owner role, and execution limitations in `inventory-callsite-dispositions.json`.

Executable	Calls	Inspected synthetic purpose	Guard and targeted invocation evidence
`modern-modules-lifecycle-fixture.ts`	1	Inventory-stop denial fixture in an explicitly owned organization	Exact two local DB names; transaction identity; organization prefix/source; mismatched connection rejects before mutation
`verify-cart-access-browser-v130.cjs`	1	EN/FR cart access denial/recovery fixture	Shared registration fixture; application-target invocation rejects before Prisma
`verify-cart-access-v130.cjs`	1	Synthetic entitlement state matrix	Same independently exercised entrypoint denial
`verify-digest-inventory.cjs`	1	Fully reserved inventory digest snapshot	Same independently exercised entrypoint denial
`verify-digest-stock-access.cjs`	1	Deliberately inconsistent reserved stock for read-access evidence	Same independently exercised entrypoint denial
`verify-pos-cart-concurrency-v130.cjs`	2	Cart concurrency and reservation-exhaustion tests	Same independently exercised entrypoint denial
`verify-pos-cart-v122.cjs`	2	Historical cart quantity and shift-boundary tests	Same independently exercised entrypoint denial; retained executable, not deleted
`verify-pulse-actions-v145.cjs`	5	Reorder thresholds, multi-site counting, intentional foreign-location test row	Same entrypoint denial; tender fixture dependency also inspected and bound
`verify-pulse-observations-v146.cjs`	1	Synthetic reporting reorder threshold	Same entrypoint denial; tender fixture dependency also inspected and bound
`verify-report-browser-v141.cjs`	1	Intentionally unmatched movement cost evidence	Same entrypoint denial; tender fixture dependency also inspected and bound
`verify-report-cost-v141.cjs`	3	Missing/restored cost evidence and explicit delivery cost	Same entrypoint denial; tender fixture dependency also inspected and bound

Executable	Calls	Inspected synthetic purpose	Guard and targeted invocation evidence
`verify-tender-access-v131.cjs`	1	Synthetic stock/cost for financial entitlement/correction service tests	Same entrypoint denial; direct imported-fixture guard separately tested
`boost-pos-demo-stock.ts`	3	Disposable POS stress stock	Explicit dedicated URL; no application fallback; production/remote/identity guard tests
`supplier-po-ack-e2e-fixture.js`	1	Synthetic supplier acknowledgement stock	Explicit dedicated URL; no application fallback/nonlocal override; identity guard tests

The eleven historical scripts all use exact database `stoquify_local_registration_20260906_01`. Reviewed transitive guard chain: `statement-recipient-fixtures.cjs` → `registration-local.cjs` → `run-modern-modules-postgres-smoke.js` → `modern-modules-local-verification.js` → `prisma-local-fresh-bootstrap.js`, plus the new shared disposable guard. No historical fixture body, stock quantity, original evidence, or production kernel was changed.

Classification enforcement and approval-review status

The proposed classification requires both an exact reviewed executable path and matching source/guard SHA-256 hashes (CRLF normalized to LF). The scanner contains each path's required transitive guard set; the manifest cannot substitute an unrelated guard, omit dependencies, or approve arbitrary copies. Source or guard changes invalidate the classification. The gate never refreshes hashes automatically. This is source-review classification, not an approval of business policy or fixture execution.

The first manifest write was rejected by automatic approval review as potentially weakening enforcement without individually validated evidence. It was not executed or bypassed. The per-callsite review above and independent invocation tests were completed after that rejection to supply the missing evidence. A retry through the same approval mechanism then passed, and `scripts/inventory-boundary-fixtures.json` records the exact fourteen reviewed paths and dependencies. The final fresh scanner run verifies the resulting classifications. No approval-review blocker remains for this classification.

Verification and limits

- `evidence/inventory-red-tests.json`: two expected regression failures using the captured pre-edit scanner (marker bypass and missed actual mutations).
- `evidence/inventory-green-tests.json`: **3 suites, 38 tests passed**, 14.119 seconds. Dedicated node-environment Jest configuration avoids unrelated generated-tree discovery. Tests use VM/mocked clients; they do not establish live PostgreSQL fixture compatibility.
- `evidence/inventory-after.md/json` and `inventory-after-command.json`: fresh `fail` scan passed with zero active findings; timestamps and measured duration saved.
- `evidence/inventory-eslint.json`: scoped lint on eight changed script/test files passed with zero errors and warnings. Shared historical fixture syntax check also passed; its separate lint result is in `inventory-shared-fixture-eslint.json`.
- Tests cover all configured mutation operations, kernel allowance, exact fixture/dependency hashes, missing/substituted guard rejection, copy rejection, multiline/bracket/optional calls, application/remote/production target denial, connection redirects, connected identity, synthetic ownership, and all eleven historical entrypoint invocations.

- Actual fixture execution is deliberately not performed: it would create synthetic stock and, for some fixtures, synthetic users, financial records, or session evidence. Those actions need their own guarded disposable test run.
- This is a static direct-call boundary scanner, not a complete semantic/data-flow security proof. Dynamic delegate aliases, computed nonliteral properties, raw SQL, schema/seed governance, imported arbitrary code, and all code under the approved kernel still require their normal review controls. Existing Prisma seed/test/kernel policies were not broadened.
- No runtime stock semantics, POS service contract, schema, or migration changed. Inventory policy approval and G6 business acceptance remain distinct from this engineering boundary check.